

Lista 03 - Introdução a Criptografia

Antônio Erick Freitas Ferreira - 322980
Instituto de Computação - Unicamp - São Paulo - Brasil

14 de abril de 2026

Questão 1

Seja k a chave descoberta e c_1 e c_2 os arquivos capturados na rede. Como $c_2 = ENC_k(0xFF \oplus IV)$, faça $y_2 = DEC_k(c_2) = 0xFF \oplus IV$. Agora extraia IV da seguinte forma: $IV = y_2 \oplus 0xFF = 0xFF \oplus IV \oplus 0xFF = IV$.

Para recuperar o conteúdo da mensagem c_1 , temos que $c_1 = ENC_k(m_1 \oplus IV)$, faça $y_1 = DEC_k(c_1) = m_1 \oplus IV$ e depois $m_1 = y_1 \oplus IV = m_1 \oplus IV \oplus IV = m_1$.

Questão 2

Item A)

Algorithm 1: Descriptador

Input: $(IV, c_1, \dots, c_l)$
Output: $m \in \{0, 1\}^{nl}$

```
1  $L \leftarrow []$ 
2 for  $1 \leq i \leq l$  do
3    $y \leftarrow F_k^{-1}(c_i)$ 
4    $x \leftarrow (y - IV - i) \bmod 2^n$ 
5   Insira  $x$  em  $L$ 
6  $m \leftarrow L_1 || L_2 || \dots || L_l$ 
7 return  $m$ 
```

Item B)

Algorithm 2: Distinguidor D

```
1  $x \leftarrow \{0, 1\}^n$ 
2  $m_0 \leftarrow (x, x - 1, x - 2, \dots, x - (l - 1)) \bmod 2^n$ 
3  $m_1 \leftarrow \{0\}^{nl}$ 
4  $(IV, c_1, c_2, \dots, c_l) \leftarrow O(m_0, m_1)$ 
5 if todos  $c_i$ , com  $1 \leq i \leq l$ , forem iguais then
6   return 0
7 else
8   return 1
```

A vantagem pode ser calculada como segue.

$$Adv = |Pr(D^O = 0 | b = 0) - Pr(D^O = 0 | b = 1)|$$

$$Pr(D^O = 0 | b = 0) = Pr(c_1 = c_2 = \dots = c_l | b = 0) = 1$$

Como $m_1 = \{0\}^{nl}$, todo c_i , com $1 \leq i \leq l$, é $F_k(IV + i)$. Como F_k é uma PRP e toda entrada é diferente, o resultado de F_k nas i consultas será diferente, então $Pr(D^O = 0 | b = 1) = 0$.

Logo,

$$Adv = |1 - 0| = 1. \text{ Como } 1 \notin \text{negl}, Adv \notin \text{negl}.$$

Questão 3

(Prova por contra-positiva) Se $\hat{\Pi}$ NÃO é CPA-Seguro, então Π NÃO é CPA-Seguro.

Desta forma, como $\hat{\Pi}$ NÃO é CPA-Seguro, existe um distinguidor $\hat{D}$ capaz de vencer o desafio CPA com vantagem não negligenciável dado c e t . Agora construiremos um distinguidor D para Π da seguinte forma.

Algorithm 3: Distinguidor D

```

1 Receba  $m_0$  e  $m_1$  de  $\hat{D}$ 
2  $c \leftarrow O((m_0, m_1))$ 
3  $t \leftarrow g(c)$ 
4 Retorne  $(c, t)$  para  $\hat{D}$ 
5 Receba  $b'$  de  $\hat{D}$ 
6 return  $b'$ 

```

A vantagem pode ser calculada como segue.

$$Adv^D = |Pr(D^O = 0|b = 0) - Pr(D^O = 1|b = 0)|$$

Como $Pr(D^O = 0|b = 0) = Pr(\hat{D}^{\hat{O}} = 0|b = 0)$ e $Pr(D^O = 1|b = 0) = Pr(\hat{D}^{\hat{O}} = 1|b = 0)$, temos que

$Adv^D = |Pr(\hat{D}^{\hat{O}} = 0|b = 0) - Pr(\hat{D}^{\hat{O}} = 1|b = 0)| = Adv^{\hat{D}}$. Como $Adv^{\hat{D}} \notin \text{negl}$ (pois $\hat{D}$ é um distinguidor assumido na hipótese), então $Adv^D \notin \text{negl}$ também.

Questão 4

Item A)

Seja $c_0 = r$ e $c_1 = (F_k(r) \oplus m)$. Uma função Dec pode ser definida como segue.

$$Dec_k(c_0, c_1) = F_k(c_0) \oplus c_1 = F_k(r) \oplus F_k(r) \oplus m = m \in \{0, 1\}^n$$

Como $\forall k \in K, \forall m \in M, Pr(Dec_k(Enc_k(m)) = m) = 1$, então Dec , como definido acima, garante corretude.

Item B)

A segurança CPA de Π se apoia no fato de F ser uma PRF . Logo, se F é uma PRF , então Π é CPA-seguro.

(Prova por contra-positiva) Se Π NÃO é CPA-seguro, então F NÃO é uma PRF .

Como Π não é CPA-seguro, então existe um distinguidor D' capaz de vencer o desafio CPA com vantagem não negligenciável dado $(r, F_k(r) \oplus m)$. Desta forma, construiremos um distinguidor D , para mostrar que F não é uma PRF , como segue.

Algorithm 4: Distinguidor D

```

1 Receba  $m_0$  e  $m_1$  de  $D'$ 
2 Seja  $b$  um bit aleatório  $\in \{0, 1\}$ 
3  $r \leftarrow \{0, 1\}^n$ 
4  $y \leftarrow O(r) \oplus m_b$ 
5 Retorne  $(r, y)$  para  $D'$ 
6 Receba  $b'$  de  $D'$ 
7 if  $b' = b$  then
8   return 1
9 else
10  return 0

```

A vantagem pode ser calculada como segue.

$$Adv = |Pr(D^{F(\cdot)} = 1|O = F) - Pr(D^{f(\cdot)} = 1|O = f)|$$

$$Pr(D^{F(\cdot)} = 1|O = F) = Pr(b' = b) = \frac{1}{2} + Adv^{D'} \text{ que } \notin \text{negl}$$

Como f é uma função aleatória, D' não sabe distinguir nesse cenário, logo $Pr(D^{f(\cdot)} = 1|O = f) = Pr(b' = b) = \frac{1}{2}$.

Portanto,

$$Adv = |(\frac{1}{2} + Adv^{D'}) - \frac{1}{2}| = Adv^{D'} \notin \text{negl}, \text{ então } F \text{ não é uma } PRF.$$

Questão 5

Item A)

Para provarmos que $(G, \oplus)$ é um grupo, precisamos provar que há associatividade, o elemento neutro e que todo elemento em G tem um inverso, isto é:

Associatividade: $\forall a, b, c \in G, (a \oplus b) \oplus c = a \oplus (b \oplus c)$

Elemento neutro: $\exists e \in G$ tal que $\forall a \in G, a \oplus e = e \oplus a = a$

Inverso: $\forall a \in G, \exists a^{-1} \in G, a \oplus a^{-1} = a^{-1} \oplus a = e$

Associatividade: Seja A, B e $C \in G$, ou seja, A, B e $C \in \{0, 1\}^n$. Como a operação $\oplus$ opera bit a bit, então temos que $(A \oplus B)_i = A_i \oplus B_i$, com $1 \leq i \leq n$. De forma análoga, temos que $((A \oplus B) \oplus C)_i = (A_i \oplus B_i) \oplus C_i$ e que $(A \oplus (B \oplus C))_i = A_i \oplus (B_i \oplus C_i)$. Como a operação $\oplus$ é associativa sobre bits ($\oplus$ sobre bits equivale a soma mod 2), temos que $(A_i \oplus B_i) \oplus C_i = A_i \oplus (B_i \oplus C_i)$. Como a igualdade é válida para todos os i bits, temos que $(A \oplus B) \oplus C = A \oplus (B \oplus C)$.

Elemento neutro: Como dito anteriormente, a operação $\oplus$ sobre bits equivale a soma mod 2. Pensando dessa forma, o elemento neutro $e = (0, 0, \dots, 0) \in G$. $\forall A \in G, A \oplus e = e \oplus A = A$, pois para todo bit A_i , com $1 \leq i \leq n$, $A_i \oplus 0 = A_i + 0 \text{ mod } 2 = A_i$ e $0 \oplus A_i = 0 + A_i \text{ mod } 2 = A_i$. Como a igualdade é válida para todos os i bits, temos que $A \oplus e = A$ e $e \oplus A = A$, portanto $\forall A \in G, A \oplus e = e \oplus A = A$.

Inverso: Ainda se valendo da ideia de que a operação $\oplus$ sobre bits equivale a soma mod 2, o inverso de qualquer elemento $A \in G$ é ele mesmo, pois para cada A_i , com $1 \leq i \leq n$, temos que $A_i \oplus A_i = A_i + A_i \text{ mod } 2 = 0$. Observe que A_i pode ser 0 ou 1, caso $A_i = 0$ temos $0 + 0 \text{ mod } 2 = 0$, caso $A_i = 1$ temos $1 + 1 \text{ mod } 2 = 0$, como as igualdades valem para todos os i bits, então, para qualquer $A \in G, A^{-1} = A$, pois $A \oplus A = e$.

Portanto, $(G, \oplus)$ é um grupo.

Item B)

Se a operação for trocada de $\oplus$ para o operador AND, $(G, \&)$ não é um grupo. Neste caso, o elemento neutro seria $e = (1, 1, \dots, 1) \in \{0, 1\}^n$. Para que qualquer $A \& A^{-1} = A^{-1} \& A = e$, com $A \in G$, isso somente seria verdade para strings da seguinte forma $A = (1, 1, \dots, 1) \in \{0, 1\}^n$, pois, se A tiver qualquer bit $A_i = 0$, não importa qual outro bit esteja sendo operado, o resultado seria 0 e como $e = (1, 1, \dots, 1)$, $A \& B \neq e$, para qualquer $B \in G$.

Questão 6

Item A)

A composição de funções é definida como segue. Para qualquer par de funções f e g , $f \circ g = f(g(x))$

Associatividade: Seja f, g e $h \in G$.

$$f \circ (g \circ h) = f(x) \circ g(h(x)) = f(g(h(x)))$$

$$(f \circ g) \circ h = f(g(x)) \circ h(x) = f(g(h(x)))$$

Como $f \circ (g \circ h) = (f \circ g) \circ h$, então o conjunto G é associativo.

Elemento neutro: Seja f e $e \in G$, tal que $e(x) = x$, isto é, e é a função identidade.

$$f \circ e = f(e(x)) = f(x)$$

$$e \circ f = e(f(x)) = f(x)$$

Como $f \circ e = e \circ f = f$, então o conjunto G possui o elemento neutro.

Inverso: Como G é o conjunto de todas as funções f tal que f é bijetora, isso quer dizer que, para qualquer $f \in G$, existe $f^{-1} \in G$.

$$f \circ f^{-1} = f(f^{-1}(x)) = e$$

$$f^{-1} \circ f = f^{-1}(f(x)) = e$$

Como as equações acima são verdade para qualquer $f \in G$, então, para qualquer função, existe uma inversa tal que, quando composto, resulta no elemento neutro de G .

Como $(G, \circ)$ é associativo, tem o elemento neutro e todo elemento tem um inverso correspondente, então $(G, \circ)$ é um grupo.

Item B)

Como $|S| = 7$ e a operação é a composição de funções, temos que $G = S_7$, isto é, G é o conjunto de todas

as permutações possíveis de 7 elementos. Então agora a questão se resume em saber se existe $g \in G$ tal que $g^7 = id$ (ou seja, g composto consigo mesmo 7 vezes tal que, na sétima vez, $g^7(n) = n$). A resposta é sim, pois como G é o conjunto de todas as permutações possíveis de $S \rightarrow S$, então existe um $g \in G$ tal que $g(n) = n + 1 \pmod{7}$. Perceba que g é um ciclo de tamanho 7 em todos os elementos de S . Portanto, $ord(g) = 7$.

Item C)

A ordem de g , como definido, é $ord(g) = n$, pois $g^t = i + t$ com $t < n$, ou seja, g^t avança uma posição no ciclo e, quando $t = n$, $g^t = 1$, isto é, retorna ao início do ciclo.

Item D)

Sabemos que $h = g^x$ e que $h(2) = 5$, então $g^x(2) = 5$. Como $g^x(i) = i + x$, temos que $g^x(2) = 2 + x$, o que implica que $2 + x = 5$, concluindo que $x = 3$. Portanto, $\log_g(h) = 3$.

Questão 7

Item A)

Como h_1 e $h_2 \in \langle g \rangle$, então $h_1 = g^x$ e $h_2 = g^y$.

Com isso, temos que $\log_g(h_1) = \log_g(g^x) = x$ e $\log_g(h_2) = \log_g(g^y) = y$.

Então, $\log_g(h_1) + \log_g(h_2) = \log_g(g^x) + \log_g(g^y) = x + y$ e $\log_g(h_1 \cdot h_2) = \log_g(g^x \cdot g^y) = \log_g(g^{x+y}) = x + y$.

Como $\log_g(h_1) + \log_g(h_2) = x + y$ e $\log_g(h_1 \cdot h_2) = x + y$, temos que $\log_g(h_1 \cdot h_2) = \log_g(h_1) + \log_g(h_2)$

Item B)

$\log_g(h_1^n) = \log_g((g^x)^n) = \log_g(g^{x \cdot n}) = x \cdot n = n \cdot x$

$n \cdot \log_g(h_1) = n \cdot \log_g(g^x) = n \cdot x$

Como $n \cdot \log_g(h_1) = n \cdot x$ e $\log_g(h_1^n) = n \cdot x$, então $n \cdot \log_g(h_1) = \log_g(h_1^n)$

Questão 8

Se $b = 0$, então escolhe $y \in G$ uniformemente e computa $c_1 = g^y$ e $c_2 = h^y$. Texto cifrado = (c_1, c_2)

Se $b = 1$, então escolhe $y, z \in G$ uniformemente e computa $c_1 = g^y$ e $c_2 = g^z$. Texto cifrado = (c_1, c_2)

Item A)

Mostrar que é possível recuperar b dado conhecimento sobre $sk = x$

Algorithm 5: Dec_k

Input: (c_1, c_2)

Output: b

```

1  $y \leftarrow c_1^x$ 
2 if  $y = c_2$  then
3   return 0
4 else
5   return 1
```

Como $h = g^x$, temos que c_2 pode assumir 2 valores, sendo $(g^x)^y$ ou $(g^x)^z$. Quando a linha 1 calcula c_1^x , o resultado é $(g^y)^x = (g^x)^y$. Desta forma, caso $y = c_2$, isto é, $(g^y)^x = (g^x)^y$, significa dizer que $b = 0$, caso contrário, $b = 1$.

Item B)

A questão basicamente pede para provar que se é DDH difícil sobre G , então é o esquema CPA-Seguro, desta forma, construirei a prova como segue.

(Prova por contra-positiva) Se o esquema não é CPA-Seguro, então o DDH é fácil sobre G . Para esta prova, assumo que a chave pública $(pk = (G, q, g, h))$ com $h = g^x$ é conhecida por todos, afinal, ela é pública.

Se o esquema não é CPA-Seguro, então quer dizer que existe um algoritmo $PPT D$ capaz de vencer o jogo CPA, isto é, D é capaz de saber qual mensagem foi cifrada ao interagir com o oráculo.

Seja A o algoritmo capaz de resolver o DDH definido como segue.

Algorithm 6: A

Input: (G, q, g, g^x, g^y, g^z)
Output: 0 se $z = xy$ ou 1 caso contrário
1 Receba m_0 e m_1 de D
2 $b \leftarrow D(g^y, g^z)$
3 **return** b

A vantagem de A pode ser calculada como segue.

$$Adv^A = |Pr(A = 0|z = xy) - Pr(A = 0|z \neq xy)|$$

Note que $Pr(A = 0|z = xy) = Pr(D = 0|b = 0)$ e $Pr(A = 0|z \neq xy) = Pr(D = 0|b = 1)$. Desta forma temos que

$$Adv^A = |Pr(D = 0|b = 0) - Pr(D = 0|b = 1)| = Adv^D$$

Como assumimos, por hipótese, que D vence o jogo CPA com vantagem não negligenciável e $Adv^A = Adv^D$, então $Adv^A \notin \text{negl}$.

Questão 9

Seja (c_1, c_2) o texto cifrado de uma mensagem m . Como $c_1 = g^r$ e $c_2 = m \cdot h^r$ com $h = g^x$, podemos definir (c'_1, c'_2) como segue.

$$c'_1 = c_1$$

$$c'_2 = c_2 \cdot u = m \cdot h^r \cdot u$$

Note que, ao descriptografar usando $Dec_k((c'_1, c'_2))$, teremos

$$Dec_k((c'_1, c'_2)) = c'_2 \cdot ((c'_1)^x)^{-1} = m \cdot (g^x)^r \cdot u \cdot ((g^r)^x)^{-1} = m \cdot u.$$

Questão 10

Prove que se DDH é difícil sobre G , então CDH também é difícil sobre G .

*Relembrando:

- DLP $\rightarrow$ dado g^x , recuperar x
- CDH $\rightarrow$ dado g^x e g^y , calcular g^{xy}
- DDH $\rightarrow$ dado (G, q, g, g^x, g^y, g^z) , descobrir se $z = xy$ ou se z é um elemento aleatório em $\mathbb{Z}_q$

(Prova por contra-positiva) Se CDH é fácil sobre G , então DDH também é fácil sobre G .

Se CDH é fácil, significa dizer que existe um algoritmo PPT A que dado g^x e g^y , retornará g^{xy} , portanto, seja D o algoritmo PPT capaz de resolver o problema DDH definido como segue. Por conveniência, assumamos que a chave pública $pk = (G, q, g, h)$, com $h = g^x$, é conhecida por todos os algoritmos, afinal, ela é pública.

Algorithm 7: D

Input: (G, q, g, g^x, g^y, g^z)
Output: 1 se $z = xy$ ou 0 caso contrário
1 $t \leftarrow A(g^x, g^y)$
2 **if** $t = g^z$ **then**
3 **return** 1
4 **else**
5 **return** 0

Assumindo que A sempre retorna g^{xy} , temos que a vantagem de D pode ser calculada como segue.

$$Adv^D = |Pr(D = 1|z = xy) - Pr(D = 1|z \leftarrow \mathbb{Z}_q)|$$

Note que $Pr(D = 1|z = xy) = 1$, pois assumimos que A tem êxito sempre, e $Pr(D = 1|z \leftarrow \mathbb{Z}_q) = \frac{1}{q}$, que é a probabilidade de haver colisão, isto é, a probabilidade de um z aleatório ser igual a xy .

Logo $Adv^D = |Pr(D = 1|z = xy) - Pr(D = 1|z \leftarrow \mathbb{Z}_q)| = |1 - \frac{1}{q}|$. Como q é a ordem de G e geralmente é um número grande, $\frac{1}{q}$ é um número próximo a 0. Portanto, $Adv^D \approx 1$ que $\notin \text{negl}$.

Questão 11

Se o melhor algoritmo para resolver o DLP em G tem complexidade $2^{(\log q)^{\frac{1}{3}}}$ e queremos ter 128 bits de segurança, façamos o seguinte

$$2^{(\log q)^{\frac{1}{3}}} = 2^{128} \rightarrow (\log q)^{\frac{1}{3}} = 128 \rightarrow \sqrt[3]{\log q} = 128 \rightarrow \log q = (2^7)^3 \rightarrow \log q = 2^{21}.$$

Portanto, a chave sk precisa ter 2^{21} bits de tamanho.

Como $h = g^x$ é um elemento no grupo, então h tem tamanho igual $10 \cdot \log q \rightarrow 10 \cdot 2^{21}$.

Os textos cifrados (c_0, c_1) ambos são pertencentes ao grupos, logo, para qualquer mensagem m , o texto cifrado terá tamanho $2 \cdot 10 \cdot \log q \rightarrow 20 \cdot 2^{21}$